

Individual Contribution Report

COU4303 Artificial Intelligence

Automated Drone Relief Delivery System | Group 45

Group number	45
Group leader	<i>V A Udara Samarasinghe - 224699029</i>
Course	COU4303 Artificial Intelligence
Project Name	Automated Drone Relief Delivery System

Summary of contributions

Member	Reg Number	Part	Line range
D.A.H.S.Pathum	124656413	Input helpers, map data, heuristic and distances	1-158
S. P. B. Udayanga	324654712	Drone rules, search algorithms, tour engine and drone state	162-295
V A Udara Samarasinghe	224699029	Option 6: multi-target tour plan (input, validation, run and fly)	297-434
V S Mediwaka	524685939	Main menu and option 8 (algorithm comparison)	436-536
K.B Yehan Bimsara	724656852	Blocked roads (options 3, 4, 5) and option 1 (view deliveries)	538-606
A.W.A.D. Thisara Dimuth Abesekara	524654961	Option 2: deliver to a selected location	608-682

Extra team duties

Duty	Member responsible
Merge the file and final full test of options 1-9	<i>Udara, Shenal</i>
Check warnings when the file loads and After test Cases	<i>Shenal, Heshan</i>
Prepare the slides (Problem, Overview, Demo, Challenges)	<i>Buddika, Yehan</i>
Check every report against the code	<i>Thisara, Yehan</i>
write the exact inputs for demo part and run them from a fresh	<i>Heshan, Buddika</i>
Verify the Contributions and Prepare the Report for Upload	<i>Udara</i>

Member 1: Input helpers, map data, heuristic and distances

Name	<i>D.A.H.S.Pathum</i>
Registration no.	<i>124656413</i>
Line range	<i>1-158 (89 code lines)</i>
Menu options / functions	<i>Used by every option (input, map, distances, A* estimates), battery/payload rules.</i>

Contribution summary

Wrote the file header, the safe input functions used by every menu option, the road map table (energy and km), the heuristic estimates that guide A*, and the function that adds up distances along a path, battery and payload rules.

Implemented functions

Function / facts	Lines	What it does
Header comments (title, how to run, program map)	1-22	Explains how to start the program and what each section contains.
safe_read/2	26-29	Reads one term from the keyboard. A syntax error becomes <code>\$invalid</code> instead of crashing the program.
resolve_var/3, find_var_name/3	32-42	An unquoted capital word (like Kandy) is a Prolog variable. These recover the name the user typed.
read_atom_input/1	45-48	Reads one location name and corrects its spelling.
canonical_location/2 , known_location/1	51-59	Matches typed text with a real map name, ignoring upper and lower case.
read_choice/1, read_answer/1	61-70	Read the menu number and the yes/no answer. A variable (typing X.) becomes <code>invalid</code> .
road/4 facts	76-88	The only map table: road(A, B, Energy, Km) for 13 roads between 7 towns.
edge/3	90-90	Gives the energy part of a road.
h/3 facts	98-128	Estimated energy from a town to a goal. Each value must be less than or equal to the real cheapest cost.
heuristic/3	131-135	Returns 0 for the same town, the stored value if there is one, and 0 otherwise.
road_km/3, path_distance_km/2	141-147	Adds up the air distance (km) of every road on a path, in either direction.

<code>full_charge_range/1,</code> <code>max_payload/1</code>	153-15 4	Full battery = 500 energy units. Maximum load = 100 kg.
<code>battery_percent/2</code>	156-15 8	Converts energy to a percentage. ceiling/1 rounds up so battery use is never under-estimated.

How it works, step by step

1. The user types an answer and a period. `safe_read` reads it with `read_term` inside `catch/3`.
2. If the word started with a capital letter, Prolog made it a variable. `resolve_var` looks it up to get the real text.
3. `canonical_location` compares the text in lower case with every map location and returns the correctly spelled name.
4. `read_choice` turns an unknown variable into `invalid`, so typing X. cannot be mistaken for option 1.
5. The map is one list of `road/4` facts. `edge/3` and `road_km/3` read from it, so a road is written only once.
6. A* adds an estimate H from `heuristic/3` to the cost so far. A missing estimate counts as 0, which is always safe.
7. `path_distance_km` walks along a path and adds the km of each road, trying both directions.

Code of this part (with line numbers)

```

1 | %----- AUTOMATED DRONE RELIEF DELIVERY SYSTEM -----
2 |
3 | % COU4303 Mini Project - route finding with DFS, BFS and A*.
4 | %
5 | % HOW TO RUN
6 | %   ?- [COU4303_Group45.pl]. (consult the file)
7 | %   ?- go.                  interactive menu
8 | % Every answer typed at a prompt must end with a period (e.g. 1.
Colombo.)
9 |

```

```

10 |
11 | % PROGRAM MAP
12 | % 1. Input helpers    safe reading of user input
13 | % 2. Map data         ONE table road/4 (energy + km) - edit only here
14 | % 3. A* heuristic     h/3 estimates + heuristic/3 wrapper
15 | % 4. Search algorithms dfs/4, bfs/4, astar/4
16 | % 5. Tour planning    multi-target tour (menu option 6)
17 | % 6. Drone state      battery, deliveries
18 | % 7. Menu and options 1-9
19 | % 8. Comparison       option 8
20 |
21 |
22 |
23 |
24 | %----- Input helpers -----
25 |
26 | safe_read(Term, Vars) :-
27 |     catch(read_term(Term, [variable_names(Vars)]),
28 |           error(syntax_error(_), _),
29 |           ( Term = '$invalid', Vars = [] ) ).
30 |
31 |
32 | resolve_var(Term, Vars, Name) :-
33 |     var(Term),
34 |     !,
35 |     ( find_var_name(Term, Vars, Found)
36 |     -> Name = Found
37 |     ; Name = Term
38 |     ).
39 | resolve_var(Term, _, Term).
40 |
41 | find_var_name(V, [N=V2|_], N) :- V == V2, !.
42 | find_var_name(V, [_|T], N) :- find_var_name(V, T, N).
43 |
44 |
45 | read_atom_input(Atom) :-
46 |     safe_read(Term, Vars),
47 |     resolve_var(Term, Vars, Name),
48 |     canonical_location(Name, Atom).
49 |
50 |
51 | canonical_location(In, Out) :-
52 |     atom(In),
53 |     downcase_atom(In, Low),
54 |     known_location(Out),
55 |     downcase_atom(Out, Low),
56 |     !.
57 | canonical_location(X, X).
58 |
59 | known_location(L) :- ( edge(L, _, _) ; edge(_, L, _) ).
60 |
61 | read_choice(Choice) :-
62 |     safe_read(Term, Vars),
63 |     resolve_var(Term, Vars, Choice0),
64 |     ( var(Choice0) -> Choice = invalid ; Choice = Choice0 ).
65 |
66 |
67 | read_answer(Answer) :-
68 |     safe_read(Term, Vars),
69 |     resolve_var(Term, Vars, Name),
70 |     ( atom(Name) -> downcase_atom(Name, Answer) ; Answer = invalid ).
71 |

```

```

72 |
73 |
74 | %----- Map data -----
75 |
76 | road('Colombo', 'Kandy', 45, 95).
77 | road('Kandy', 'Badulla', 27, 58).
78 | road('Colombo', 'Badulla', 68, 134).
79 | road('Colombo', 'Ratnapura', 32, 67).
80 | road('Ratnapura', 'Badulla', 38, 79).
81 | road('Ratnapura', 'Galle', 36, 74).
82 | road('Ratnapura', 'Hambantota', 43, 91).
83 | road('Kandy', 'Polonnaruwa', 41, 83).
84 | road('Colombo', 'Galle', 53, 106).
85 | road('Hambantota', 'Galle', 50, 101).
86 | road('Badulla', 'Hambantota', 48, 96).
87 | road('Badulla', 'Polonnaruwa', 53, 106).
88 | road('Kandy', 'Ratnapura', 37, 72).
89 |
90 | edge(A, B, Energy) :- road(A, B, Energy, _).
91 |
92 |
93 |
94 | %----- A* heuristic values -----
95 |
96 | % h(Node, Goal, Estimate).
97 |
98 | h('Ratnapura', 'Hambantota', 43).
99 | h('Galle', 'Hambantota', 49).
100 | h('Badulla', 'Hambantota', 47).
101 |
102 | h('Ratnapura', 'Colombo', 31).
103 | h('Kandy', 'Colombo', 44).
104 | h('Badulla', 'Colombo', 67).
105 | h('Galle', 'Colombo', 52).
106 |
107 | h('Kandy', 'Badulla', 26).
108 | h('Colombo', 'Badulla', 67).
109 | h('Ratnapura', 'Badulla', 37).
110 | h('Polonnaruwa', 'Badulla', 52).
111 |
112 | h('Colombo', 'Ratnapura', 31).
113 | h('Badulla', 'Ratnapura', 37).
114 | h('Galle', 'Ratnapura', 35).
115 | h('Hambantota', 'Ratnapura', 42).
116 | h('Kandy', 'Ratnapura', 36).
117 |
118 | h('Ratnapura', 'Kandy', 36).
119 | h('Colombo', 'Kandy', 44).
120 | h('Badulla', 'Kandy', 26).
121 | h('Polonnaruwa', 'Kandy', 40).
122 |
123 | h('Kandy', 'Polonnaruwa', 40).
124 | h('Badulla', 'Polonnaruwa', 52).
125 |
126 | h('Colombo', 'Galle', 52).
127 | h('Hambantota', 'Galle', 49).
128 | h('Ratnapura', 'Galle', 35).
129 |
130 |
131 | heuristic(A, B, H) :-
132 |     ( A == B -> H = 0
133 |     ; h(A, B, H0) -> H = H0

```

```

134 |      ;   H = 0
135 |      ).
136 |
137 |
138 |
139 | %----- Road distances in kilometres -----
140 |
141 | road_km(A, B, Km) :- road(A, B, _, Km).
142 |
143 | path_distance_km([_], 0).
144 | path_distance_km([A, B | Rest], Km) :-
145 |     (   road_km(A, B, Leg) ; road_km(B, A, Leg) ), !,
146 |     path_distance_km([B | Rest], RestKm),
147 |     Km is Leg + RestKm.
148 |
149 |

```

Real output: wrong menu option (typing x.)

```
Invalid option, please try again.
```

Real output: lower-case name accepted (option 2, kandy.)

```

--- Delivery Details (round trip) ---
Location: Kandy
Distance: 190 km (round trip)
Route: [Colombo,Kandy]
Battery Cost: 18% (round trip)
Package Weight: 20kg
Battery Available: 100%
Battery After Delivery: 82%

Confirm this delivery? (yes. / no.):
Delivery cancelled. No battery used.

```

Proof that every heuristic value is safe ($h \leq$ real cheapest cost)

Goal	From	h (estimate)	Real cheapest cost	$h \leq$ real ?
Badulla	Colombo	67	68	yes
Badulla	Kandy	26	27	yes
Badulla	Polonnaruwa	52	53	yes
Badulla	Ratnapura	37	38	yes
Colombo	Badulla	67	68	yes
Colombo	Galle	52	53	yes
Colombo	Kandy	44	45	yes
Colombo	Ratnapura	31	32	yes
Galle	Colombo	52	53	yes
Galle	Hambantota	49	50	yes

Galle	Ratnapura	35	36	yes
Hambantota	Badulla	47	48	yes
Hambantota	Galle	49	50	yes
Hambantota	Ratnapura	43	43	yes
Kandy	Badulla	26	27	yes
Kandy	Colombo	44	45	yes
Kandy	Polonnaruwa	40	41	yes
Kandy	Ratnapura	36	37	yes
Polonnaruwa	Badulla	52	53	yes
Polonnaruwa	Kandy	40	41	yes
Ratnapura	Badulla	37	38	yes
Ratnapura	Colombo	31	32	yes
Ratnapura	Galle	35	36	yes
Ratnapura	Hambantota	42	43	yes
Ratnapura	Kandy	36	37	yes

Member 2: Drone rules, search algorithms, tour engine and drone state

Name	<i>S. P. B. Udayanga</i>
Registration no.	324654712
Line range	164-295 (92 code lines)
Menu options / functions	DFS, BFS, A*; engine used by option 6; battery and delivered records

Contribution summary

Wrote the blocked-road and delivery-point facts, the move rule used by every search, the three search algorithms (DFS, BFS, A*), the tour planning engine and the drone state (location, battery, delivered camps).

Implemented functions

Function / facts	Lines	What it does
<code>blocked/2 (dynamic)</code>	164-165	Blocked roads. Starts with Ratnapura-Kandy (bad weather).
<code>delivery_point/2</code>	171-176	The six camps with package weights.
<code>valid_move/3</code>	178-181	A move is allowed when a road exists in either direction and it is not blocked in either direction.
<code>dfs/4, dfs_helper/5</code>	187-196	Depth-first search with a visited list. Backtracking gives every possible path.
<code>bfs/4, bfs_queue/4</code>	199-211	Breadth-first search with a queue. Finds the route with the fewest roads.
<code>astar/4, astar_search/4</code>	214-230	A* search. Returns the path and its energy cost, guided by heuristic/3.
<code>run_algo/5</code>	232-234	Runs dfs, bfs or astar and keeps only the first answer using once/1.
<code>plan_tour/5, plan_legs/5</code>	240-249	Builds the route Start -> targets -> Start one leg at a time and adds the costs.
<code>combine_paths/3</code>	251-252	Joins two paths without repeating the town where they meet.
<code>best_tour_order/5</code>	255-267	Tries every visiting order with A* and keeps the cheapest one (up to 6 targets).

<code>max_battery/1</code> and <code>dynamic facts</code>	273-277	<code>current_location/1</code> , <code>battery_level/1</code> and <code>delivered/1</code> change while the program runs.
<code>reset_drone/0</code> , <code>init_drone/0</code>	279-289	<code>reset_drone</code> puts the drone at Colombo with 100% battery. <code>init_drone</code> also clears the delivered list.
<code>available_location/2</code>	291-293	A delivery point that has not been delivered yet.

How it works, step by step

1. `valid_move` is the gate for every search: a road is used only if it exists and is not blocked in either direction.
2. DFS goes to a neighbour not yet visited; at the target the cost is 0 and each step adds its cost while returning.
3. BFS keeps a queue of [`PathBackwards`, `Cost`]; children go to the END of the queue, so shorter paths are tried first.
4. A* keeps a list of [`F`, `G`, `Path`], expands the smallest `F` first (sort/2) and stops when the first path ends at the target.
5. `plan_tour` appends the base to the target list; `plan_legs` calls `run_algo` for each leg and `combine_paths` joins the legs.
6. `best_tour_order` uses `permutation/2`, collects `Cost-Order-Path` and `keysort` puts the cheapest first.
7. `init_drone` clears delivered camps and calls `reset_drone`, which sets Colombo and 100% battery.

Code of this part (with line numbers)

```
151 | %----- Battery and payload -----
152 |
153 | full_charge_range(500).
154 | max_payload(100).
155 |
156 | battery_percent(Raw, Pct) :-
157 |     full_charge_range(Range),
158 |     Pct is ceiling(Raw * 100 / Range).
159 |
160 |
161 |
162 | %----- No-fly zones -----
163 |
164 | :- dynamic(blocked/2).
165 | blocked('Ratnapura', 'Kandy'). % Bad weather.
166 |
167 |
168 |
169 | %----- Delivery points and package weights -----
170 |
171 | delivery_point('Hambantota', 15).
172 | delivery_point('Badulla', 15).
173 | delivery_point('Kandy', 20).
174 | delivery_point('Polonnaruwa', 15).
175 | delivery_point('Galle', 15).
176 | delivery_point('Ratnapura', 20).
177 |
178 | valid_move(Current, Next, Cost) :-
179 |     (edge(Current, Next, Cost) ; edge(Next, Current, Cost)),
180 |     \+ blocked(Current, Next),
181 |     \+ blocked(Next, Current).
182 |
183 |
184 |
185 | %----- Search algorithms -----
186 |
187 | dfs(Start, Target, Path, Cost) :-
188 |     dfs_helper(Start, Target, [Start], RevPath, Cost),
189 |     reverse(RevPath, Path).
190 |
191 | dfs_helper(Target, Target, Visited, Visited, 0).
192 | dfs_helper(Current, Target, Visited, FinalPath, TotalCost) :-
193 |     valid_move(Current, Next, StepCost),
194 |     \+ member(Next, Visited),
195 |     dfs_helper(Next, Target, [Next|Visited], FinalPath, RestCost),
196 |     TotalCost is StepCost + RestCost.
197 |
198 |
199 | bfs(Start, Target, Path, Cost) :-
200 |     bfs_queue([ [Start], 0 ], Target, RevPath, Cost),
201 |     reverse(RevPath, Path).
202 |
203 | bfs_queue([ [Target|Rest], Cost ] | _ , Target, [Target|Rest], Cost).
204 | bfs_queue([ [Current|Rest], Cost ] | QueueTail , Target, FinalPath,
FinalCost) :-
205 |     findall(
206 |         [[Next, Current|Rest], NewCost],
207 |         (valid_move(Current, Next, StepCost), \+ member(Next,
[Current|Rest]), NewCost is Cost + StepCost),
208 |         Children
```

```

209 |     ),
210 |     append(QueueTail, Children, NewQueue),
211 |     bfs_queue(NewQueue, Target, FinalPath, FinalCost).
212 |
213 |
214 | astar(Start, Target, Path, Cost) :-
215 |     heuristic(Start, Target, H),
216 |     astar_search([[H,0,[Start]]], Target, RevPath, Cost),
217 |     reverse(RevPath, Path).
218 |
219 | astar_search([[_ ,Cost,[Target|Rest]]|_], Target, [Target|Rest], Cost).
220 | astar_search([[_ ,G,[Current|Rest]]|Others], Target, Path, Cost) :-
221 |     findall([F2, G2, [Next,Current|Rest]],
222 |         (   valid_move(Current, Next, StepCost),
223 |             \+ member(Next, [Current|Rest]),
224 |             G2 is G + StepCost,
225 |             heuristic(Next, Target, H),
226 |             F2 is G2 + H),
227 |         Children),
228 |     append(Others, Children, All),
229 |     sort(All, SortedQueue),
230 |     astar_search(SortedQueue, Target, Path, Cost).
231 |
232 | run_algo(dfs, S, T, P, C) :- once(dfs(S, T, P, C)).
233 | run_algo(bfs, S, T, P, C) :- once(bfs(S, T, P, C)).
234 | run_algo(astar, S, T, P, C) :- once(astar(S, T, P, C)).
235 |
236 |
237 |
238 | %----- Multi-target tour -----
239 |
240 | plan_tour(Start, Targets, Algo, FullPath, TotalCost) :-
241 |     append(Targets, [Start], FullRoute),
242 |     plan_legs(Start, FullRoute, Algo, FullPath, TotalCost).
243 |
244 | plan_legs(_, [], _, [], 0).
245 | plan_legs(Current, [NextTarget|Rest], Algo, Path, TotalCost) :-
246 |     run_algo(Algo, Current, NextTarget, LegPath, LegCost),
247 |     plan_legs(NextTarget, Rest, Algo, RestPath, RestCost),
248 |     TotalCost is LegCost + RestCost,
249 |     combine_paths(LegPath, RestPath, Path).
250 |
251 | combine_paths(P1, [], P1) :- !.
252 | combine_paths(P1, [_|T2], Combined) :- append(P1, T2, Combined).
253 |
254 |
255 | best_tour_order(Start, Targets, BestOrder, BestPath, BestCost) :-
256 |     length(Targets, N),
257 |     (   N <= 6
258 |     -> findall(Cost-Order-Path,
259 |         (   permutation(Targets, Order),
260 |             plan_tour(Start, Order, astar, Path, Cost)
261 |         ),
262 |         Plans),
263 |     Plans \== [],
264 |     keysort(Plans, [BestCost-BestOrder-BestPath | _])
265 |     ;   plan_tour(Start, Targets, astar, BestPath, BestCost),
266 |     BestOrder = Targets
267 |     ).
268 |
269 |
270 |

```

```

271 | %----- Current drone state -----
272 |
273 | max_battery(100).
274 |
275 | :- dynamic(current_location/1).
276 | :- dynamic(battery_level/1).
277 | :- dynamic(delivered/1). % Served locations.
278 |
279 | reset_drone :-
280 |     max_battery(Max),
281 |     retractall(current_location(_)),
282 |     retractall(battery_level(_)),
283 |     assertz(current_location('Colombo')),
284 |     assertz(battery_level(Max)).
285 |
286 |
287 | init_drone :-
288 |     retractall(delivered(_)),
289 |     reset_drone.
290 |
291 | available_location(Loc, Weight) :-
292 |     delivery_point(Loc, Weight),
293 |     \+ delivered(Loc).
294 |
295 |

```

Real output: DFS, BFS and A* from Colombo to Hambantota

```

--- DFS, BFS and A* : Colombo to Hambantota ---
All possible loop-free paths (15 found by DFS):
  cost 120 : [Colombo,Kandy,Badulla,Hambantota]
  cost 196 : [Colombo,Kandy,Badulla,Ratnapura,Galle,Hambantota]
  cost 153 : [Colombo,Kandy,Badulla,Ratnapura,Hambantota]
  ...

dfs  roads used: 3 | energy: 120 | distance: 249 km | battery: 24% | search
effort: 50 operations
    route: [Colombo,Kandy,Badulla,Hambantota]
bfs  roads used: 2 | energy: 116 | distance: 230 km | battery: 24% | search
effort: 565 operations
    route: [Colombo,Badulla,Hambantota]
astar roads used: 2 | energy: 75 | distance: 158 km | battery: 15% | search
effort: 208 operations
    route: [Colombo,Ratnapura,Hambantota]

Lowest energy cost (75) : [astar]
Fewest roads used (2)   : [bfs,astar]

```

Real output: best visiting order for the tour Kandy, Badulla

```

--- Best Visiting Order (A* legs, all orders tried) ---
Visit order: [Badulla,Kandy]
Full Route: [Colombo,Badulla,Kandy,Colombo]
Distance: 287 km | Battery Cost: 28% | Total Weight: 35kg
Battery Available: 100%
Battery After Tour: 72%

```

Member 3: Option 6: multi-target tour plan (input, validation, run and fly)

Name	<i>V A U Samarasinghe</i>
Registration no.	224699029
Line range	297-434 (119 code lines)
Menu options / functions	Option 6; Getting destinations, checking payload limits, finding the best order using DFS, BFS, A*, verifying battery levels, and updating the system state.

Contribution summary

The whole of option 6: reading the list of targets, checking the tour is valid, comparing DFS, BFS and A*, choosing the best visiting order, asking for confirmation and flying the tour.

Implemented functions

Function / facts	Lines	What it does
<code>plan_tour_menu/0</code>	299-315	Option 6 entry: shows available camps and the payload limit, reads the targets.
<code>read_targets/1</code>	319-323	Reads a comma list such as Kandy, Badulla.
<code>resolve_tuple/3,</code> <code>tuple_to_list/2</code>	326-342	Turn the comma term (Kandy, Badulla) into the list [Kandy, Badulla].
<code>validate_targets/2</code>	344-365	Removes duplicates, rejects wrong or served camps, rejects a total weight above 100 kg.
<code>tour_weight/2</code>	367-369	Adds the package weights of the targets.

<code>run_tour/3</code>	372-384	Shows the comparison, shows the best tour, asks for confirmation and flies it.
<code>show_tour_comparison/3</code>	387-392	Prints DFS, BFS and A* for the targets in the order entered.
<code>show_best_tour/6</code>	395-411	Shows the cheapest order and checks the battery; prints "TOUR DENIED" when it is not enough.
<code>report_tour/4</code>	413-421	One line for an algorithm with distance, battery cost and OK / NOT ENOUGH BATTERY.
<code>execute_tour/4</code>	423-433	Reduces the battery and marks every target as delivered.

How it works, step by step

1. `read_targets` reads one comma term; `resolve_tuple` fixes capital words inside it and `tuple_to_list` splits it into a list.
2. `validate_targets` removes duplicates, checks every target is an unserved delivery point and checks the weight against `max_payload`.
3. If validation fails it prints the reason and fails, so the tour is not planned.
4. `run_tour` first prints the DFS, BFS and A* table for the entered order (`show_tour_comparison`).
5. `show_best_tour` asks `best_tour_order` (Member 2) for the cheapest order and prints route, distance, battery and weight.
6. If the battery percentage is within the remaining battery, `ask_confirmation` (Member 6) is called; otherwise "TOUR DENIED" is printed.

7. `execute_tour` swaps the battery fact (retract then `assertz`) and adds `delivered(T)` for each camp.

Code of this part (with line numbers)

```
297 | %----- Option 6: Multi-target tour -----
298 |
299 | plan_tour_menu :-
300 |     current_location(Start),
301 |     battery_level(Bat),
302 |     nl, write('--- Multi-Target Tour Planning ---'), nl,
303 |     findall(L, available_location(L, _), Available),
304 |     ( Available == []
305 |     -> write('All relief camps have been served. Nothing left to
deliver.'), nl
306 |     ; format('Available locations: ~w~n', [Available]),
307 |     max_payload(Payload),
308 |     format('Drone payload limit: ~wkg~n', [Payload]),
309 |     write('Enter targets separated by commas and end with a period
(e.g., Kandy, Badulla.): '),
310 |     read_targets(RawTargets),
311 |     ( validate_targets(RawTargets, Targets)
312 |     -> run_tour(Start, Targets, Bat)
313 |     ; true
314 |     )
315 |     ).
316 |
317 |
318 |
319 | read_targets(Targets) :-
320 |     safe_read(Term, Vars),
321 |     resolve_tuple(Term, Vars, Resolved),
322 |     tuple_to_list(Resolved, Targets0),
323 |     maplist(Canonical_location, Targets0, Targets).
324 |
325 |
326 | resolve_tuple(Term, Vars, Resolved) :-
327 |     ( var(Term)
328 |     -> resolve_var(Term, Vars, Resolved)
329 |     ; Term = (A, B)
330 |     -> resolve_tuple(A, Vars, RA),
331 |     resolve_tuple(B, Vars, RB),
332 |     Resolved = (RA, RB)
333 |     ; Resolved = Term
334 |     ).
335 |
336 | tuple_to_list((A, B), [A | Rest]) :-
337 |     !, tuple_to_list(B, Rest).
338 | tuple_to_list(A, List) :-
339 |     ( ( A == end_of_file ; A == '$invalid' ; A == '' )
340 |     -> List = []
341 |     ; List = [A]
342 |     ).
343 |
344 | validate_targets([], _) :-
345 |     !, nl, write('No valid targets specified.'), nl, fail.
```

```

346 | validate_targets(Raw, Targets) :-
347 |     list_to_set(Raw, Targets),
348 |     ( member(T, Targets), \+ ( atom(T), available_location(T, _) )
349 |     -> nl,
350 |     ( atom(T), delivered(T)
351 |     -> format('~w has already received its delivery.~n', [T])
352 |     ; format('~w is not a valid delivery location.~n', [T])
353 |     ),
354 |     fail
355 |     ; true
356 |     ),
357 |     tour_weight(Targets, Weight),
358 |     max_payload(MaxPayload),
359 |     ( Weight =< MaxPayload
360 |     -> true
361 |     ; nl, write('*** TOUR DENIED ***'), nl,
362 |     format('Total package weight ~wkg exceeds the drone payload limit
of ~wkg.~n',
363 |           [Weight, MaxPayload]),
364 |     fail
365 |     ).
366 |
367 | tour_weight(Targets, Total) :-
368 |     findall(W, ( member(T, Targets), delivery_point(T, W) ), Ws),
369 |     sum_list(Ws, Total).
370 |
371 |
372 | run_tour(Start, Targets, Bat) :-
373 |     show_tour_comparison(Start, Targets, Bat),
374 |     ( show_best_tour(Start, Targets, Bat, BestOrder, BestPath, Pct)
375 |     -> ( Pct =< Bat
376 |     -> nl,
377 |         ( ask_confirmation('Fly this tour now? (yes. / no.): ')
378 |         -> execute_tour(Start, BestOrder, BestPath, Pct)
379 |         ; nl, write('Tour cancelled. No battery used.'), nl
380 |         )
381 |         ; true
382 |         )
383 |         ; write('No valid tour found (roads may be blocked).'), nl
384 |         ).
385 |
386 |
387 | show_tour_comparison(Start, Targets, Bat) :-
388 |     nl, write('--- Comparing Algorithms for the Tour (order as entered
---'), nl,
389 |     format('Visit order: ~w, then return to ~w~n', [Targets, Start]),
390 |     report_tour(dfs, Start, Targets, Bat),
391 |     report_tour(bfs, Start, Targets, Bat),
392 |     report_tour(aster, Start, Targets, Bat).
393 |
394 |
395 | show_best_tour(Start, Targets, Bat, BestOrder, BestPath, Pct) :-
396 |     nl, write('--- Best Visiting Order (A* legs, all orders tried
---'), nl,
397 |     best_tour_order(Start, Targets, BestOrder, BestPath, BestCost),
398 |     battery_percent(BestCost, Pct),
399 |     path_distance_km(BestPath, Km),
400 |     tour_weight(BestOrder, Weight),
401 |     format('Visit order: ~w~n', [BestOrder]),
402 |     format('Full Route: ~w~n', [BestPath]),
403 |     format('Distance: ~w km | Battery Cost: ~w% | Total Weight:
~wkg~n', [Km, Pct, Weight]),

```

```

404 |     format('Battery Available: ~w%~n', [Bat]),
405 |     (   Pct =< Bat
406 |     -> Remaining is Bat - Pct,
407 |     format('Battery After Tour: ~w%~n', [Remaining])
408 |     ;   nl, write('*** TOUR DENIED ***'), nl,
409 |     format('Insufficient battery for this tour. Required: ~w%,
Available: ~w%~n', [Pct, Bat]),
410 |     write('Try fewer targets, or reset the drone (option 7).'), nl
411 |     ).
412 |
413 | report_tour(Algo, Start, Targets, Bat) :-
414 |     (   plan_tour(Start, Targets, Algo, Path, Cost)
415 |     -> battery_percent(Cost, Pct),
416 |     path_distance_km(Path, Km),
417 |     (   Pct =< Bat -> Status = 'OK' ; Status = 'NOT ENOUGH BATTERY' ),
418 |     format('~w~t~7| Distance: ~w km | Battery Cost: ~w% (~w)~n    Full
Route: ~w~n',
419 |           [Algo, Km, Pct, Status, Path])
420 |     ;   format('~w~t~7| No valid tour found.~n', [Algo])
421 |     ).
422 |
423 | execute_tour(Start, Order, Path, Pct) :-
424 |     battery_level(Bat),
425 |     NewBat is Bat - Pct,
426 |     retract(battery_level(Bat)),
427 |     assertz(battery_level(NewBat)),
428 |     forall(member(T, Order), assertz(delivered(T))),
429 |     tour_weight(Order, Weight),
430 |     nl, write('*** TOUR APPROVED ***'), nl,
431 |     format('Route Taken: ~w~n', [Path]),
432 |     format('Delivered ~wkg of supplies to ~w and returned to ~w.~n',
[Weight, Order, Start]),
433 |     format('Battery Used: ~w%    |    Battery Remaining: ~w%~n', [Pct,
NewBat]).
434 |

```

Real output: option 6 with Kandy, Badulla (comparison and best order)

```

--- Comparing Algorithms for the Tour (order as entered) ---
Visit order: [Kandy,Badulla], then return to Colombo
dfs      Distance: 491 km | Battery Cost: 48% (OK)
      Full Route: [Colombo,Kandy,Badulla,Hambantota,Galle,Ratnapura,Colombo]
bfs      Distance: 287 km | Battery Cost: 28% (OK)
      Full Route: [Colombo,Kandy,Badulla,Colombo]
astar    Distance: 287 km | Battery Cost: 28% (OK)
      Full Route: [Colombo,Kandy,Badulla,Colombo]

--- Best Visiting Order (A* legs, all orders tried) ---
Visit order: [Badulla,Kandy]
Full Route: [Colombo,Badulla,Kandy,Colombo]
Distance: 287 km | Battery Cost: 28% | Total Weight: 35kg
Battery Available: 100%
Battery After Tour: 72%

```

Real output: after answering yes

```

*** TOUR APPROVED ***
Route Taken: [Colombo,Badulla,Kandy,Colombo]
Delivered 35kg of supplies to [Badulla,Kandy] and returned to Colombo.
Battery Used: 28%    |    Battery Remaining: 72%

```

Real output: a tour denied (battery 50%, needs 53%)

```
--- Best Visiting Order (A* legs, all orders tried) ---  
Visit order: [Polonnaruwa,Hambantota,Ratnapura]  
Full Route: [Colombo,Kandy,Polonnaruwa,Badulla,Hambantota,Ratnapura,Colombo]  
Distance: 538 km | Battery Cost: 53% | Total Weight: 50kg  
Battery Available: 50%
```

```
*** TOUR DENIED ***  
Insufficient battery for this tour. Required: 53%, Available: 50%  
Try fewer targets, or reset the drone (option 7).
```

Real output: a wrong target (Colombo is the base)

```
Available locations: [Hambantota,Badulla,Kandy,Polonnaruwa,Galle,Ratnapura]  
Drone payload limit: 100kg  
Enter targets separated by commas and end with a period (e.g., Kandy,  
Badulla.):  
Colombo is not a valid delivery location.
```

Member 4: Main menu and option 8 (algorithm comparison)

Name	<i>V S Mediwaka</i>
Registration no.	524685939
Line range	436-536 (85 code lines)
Menu options / functions	Menu loop, options 7 and 9, option 8

Contribution summary

the main menu that starts every option (including reset and exit) and option 8, which compares DFS, BFS and A* for any two locations.

Implemented functions

Function / facts	Lines	What it does
------------------	-------	--------------

<code>go/0</code>	438-44 0	Starts a new session with <code>init_drone</code> and shows the menu.
<code>menu_loop/0</code>	442-46 1	Prints the menu and the battery, reads the choice and calls <code>handle_choice</code> .
<code>handle_choice/1</code>	463-47 4	One clause per menu number. Every option except 9 calls <code>menu_loop</code> again.
<code>compare_menu/0,</code> <code>write_known_locations/0</code>	480-49 4	Option 8: asks for a start and a goal and checks both are real locations.
<code>compare_algorithms/2</code>	497-51 9	Lists every DFS path, then prints one line per algorithm.
<code>measure_algo/6</code>	521-52 5	Counts how many Prolog operations an algorithm used (search effort).
<code>summarise_comparison/1</code>	527-53 4	Shows which algorithm has the lowest energy and which uses the fewest roads.

How it works, step by step

1. `go` calls `init_drone`, then `menu_loop` prints the menu and calls `read_choice` (Member 1).
2. `handle_choice(N)` runs the option and then calls `menu_loop` again. This recursion is the loop.
3. The cut `!` in each clause stops Prolog from also trying the "Invalid option" clause. Option 9 does not call `menu_loop`, so it ends the program.
4. Option 8: `findall` collects every DFS path, `measure_algo` runs each algorithm and counts inferences with `statistics/2`.

5. summarise_comparison uses min_list to find the lowest energy and the fewest roads and prints which algorithms achieved them.

Code of this part (with line numbers)

```
436 | %----- Delivery menu -----
437 |
438 | go :-
439 |     init_drone,
440 |     menu_loop.
441 |
442 | menu_loop :-
443 |     nl, write('*****'), nl,
444 |     write('        DRONE RELIEF DELIVERY - MAIN MENU'), nl,
445 |     write('*****'), nl,
446 |     write('-----'), nl,
447 |     current_location(Loc), battery_level(Bat),
448 |     format('Current Location: ~w | Battery Remaining: ~w%\n', [Loc,
Bat]),
449 |     write('-----'), nl,
450 |     write('1. View delivery locations (distance/route/battery
cost/weight)'), nl,
451 |     write('2. Deliver to a selected location'), nl,
452 |     write('3. Block a route'), nl,
453 |     write('4. Unblock a route'), nl,
454 |     write('5. Show blocked routes'), nl,
455 |     write('6. Plan a Multi-Target Tour'), nl,
456 |     write('7. Reset drone (system reset)'), nl,
457 |     write('8. Compare DFS / BFS / A* between two locations'), nl,
458 |     write('9. Exit'), nl,
459 |     write('Select an option (end with a period, e.g. 1.): '),
460 |     read_choice(Choice),
461 |     handle_choice(Choice).
462 |
463 | handle_choice(1) :- !, view_delivery_details, menu_loop.
464 | handle_choice(2) :- !, deliver_to_location, menu_loop.
465 | handle_choice(3) :- !, block_road, menu_loop.
466 | handle_choice(4) :- !, unblock_road, menu_loop.
467 | handle_choice(5) :- !, show_blocked, menu_loop.
468 | handle_choice(6) :- !, plan_tour_menu, menu_loop.
469 | handle_choice(7) :- !, init_drone, nl,
470 |     write('Drone reset: at Colombo, battery 100%, all deliveries
cleared.'), nl, menu_loop.
471 | handle_choice(8) :- !, compare_menu, menu_loop.
472 | handle_choice(9) :- !, nl, write('Exiting Drone Relief Delivery System.
Safe travels!'), nl.
473 | handle_choice(end_of_file) :- !, nl, write('Exiting Drone Relief
Delivery System.'), nl.
474 | handle_choice(_) :- nl, write('Invalid option, please try again.'), nl,
menu_loop.
475 |
476 |
477 |
478 | %----- Option 8: compare the three algorithms -----
479 |
480 | compare_menu :-
481 |     nl, write('--- Algorithm Comparison ---'), nl,
```

```

482 |     write('Enter start location (end with a period, e.g. Ratnapura.):
'|),
483 |     read_atom_input(Start),
484 |     write('Enter goal location (end with a period, e.g. Hambantota.):
'|),
485 |     read_atom_input(Goal),
486 |     (    var(Start) ; var(Goal) ; \+ known_location(Start) ; \+
known_location(Goal) )
487 |     -> nl, write('Unknown location. Known locations: '),
write_known_locations, nl
488 |     ;    compare_algorithms(Start, Goal)
489 |     ).
490 |
491 | write_known_locations :-
492 |     findall(L, known_location(L), All),
493 |     sort(All, Sorted),
494 |     write(Sorted).
495 |
496 |
497 | compare_algorithms(Start, Goal) :-
498 |     nl, format('--- DFS, BFS and A* : ~w to ~w ---~n', [Start, Goal]),
499 |     findall(C-P, dfs(Start, Goal, P, C), AllPaths),
500 |     (    AllPaths == []
501 |     -> write('No path found (roads may be blocked).'), nl
502 |     ;    length(AllPaths, N),
503 |     format('All possible loop-free paths (~w found by DFS):~n', [N]),
504 |     forall(member(C-P, AllPaths),
505 |             format('    cost ~w : ~w~n', [C, P])),
506 |     nl,
507 |     findall(Algo-Cost-Roads,
508 |             ( member(Algo, [dfs, bfs, astar]),
509 |             measure_algo(Algo, Start, Goal, Path, Cost, Work),
510 |             length(Path, Len), Roads is Len - 1,
511 |             path_distance_km(Path, Km),
512 |             battery_percent(Cost, Pct),
513 |             format('~w~t~6| roads used: ~w | energy: ~w | distance:
~w km | battery: ~w% | search effort: ~w operations~n',
514 |                 [Algo, Roads, Cost, Km, Pct, Work])),
515 |             format('    route: ~w~n', [Path])
516 |             ),
517 |             Results),
518 |     summarise_comparison(Results)
519 |     ).
520 |
521 | measure_algo(Algo, Start, Goal, Path, Cost, Work) :-
522 |     statistics(inferences, I0),
523 |     run_algo(Algo, Start, Goal, Path, Cost),
524 |     statistics(inferences, I1),
525 |     Work is I1 - I0.
526 |
527 | summarise_comparison(Results) :-
528 |     findall(C, member(_-C-, Results), Costs), min_list(Costs, MinC),
529 |     findall(L, member(_-L-, Results), RoadCounts),
min_list(RoadCounts, MinL),
530 |     findall(A, member(A-MinC-, Results), CheapAlgos),
531 |     findall(A, member(A-MinL-, Results), ShortAlgos),
532 |     nl,
533 |     format('Lowest energy cost (~w) : ~w~n', [MinC, CheapAlgos]),
534 |     format('Fewest roads used (~w) : ~w~n', [MinL, ShortAlgos]).
535 |
536 |

```

Real output: the main menu

```
*****
      DRONE RELIEF DELIVERY - MAIN MENU
*****
-----
Current Location: Colombo   |   Battery Remaining: 100%
-----
1. View delivery locations (distance/route/battery cost/weight)
2. Deliver to a selected location
3. Block a route
4. Unblock a route
5. Show blocked routes
6. Plan a Multi-Target Tour
7. Reset drone (system reset)
8. Compare DFS / BFS / A* between two locations
9. Exit
Select an option (end with a period, e.g. 1.):
```

Real output: option 8, Colombo to Hambantota

```
--- DFS, BFS and A* : Colombo to Hambantota ---
All possible loop-free paths (15 found by DFS):
  cost 120 : [Colombo,Kandy,Badulla,Hambantota]
  cost 196 : [Colombo,Kandy,Badulla,Ratnapura,Galle,Hambantota]
  cost 153 : [Colombo,Kandy,Badulla,Ratnapura,Hambantota]
  ...

dfs  roads used: 3 | energy: 120 | distance: 249 km | battery: 24% | search
effort: 50 operations
    route: [Colombo,Kandy,Badulla,Hambantota]
bfs  roads used: 2 | energy: 116 | distance: 230 km | battery: 24% | search
effort: 565 operations
    route: [Colombo,Badulla,Hambantota]
astar roads used: 2 | energy: 75 | distance: 158 km | battery: 15% | search
effort: 208 operations
    route: [Colombo,Ratnapura,Hambantota]

Lowest energy cost (75) : [astar]
Fewest roads used (2)   : [bfs,astar]
```

Member 5: Blocked roads (options 3, 4, 5) and option 1 (view deliveries)

Name	<i>K B Yehan Bimsara</i>
Registration no.	724656852
Line range	538-606 (57 code lines)
Menu options / functions	Options 1, 3, 4, 5

Contribution summary

the options that block, unblock and list roads, and option 1, which shows the round-trip cost of every camp that still needs a delivery.

Implemented functions

Function / facts	Lines	What it does
<code>block_road/0,</code> <code>known_road/2</code>	540-55 5	Reads two towns, checks the road exists and is not blocked already, then blocks it.
<code>unblock_road/0</code>	557-56 8	Removes the block in both directions with retractall.
<code>show_blocked/0,</code> <code>list_blocked/0</code>	570-57 4	Lists the blocked roads or prints "None".
<code>show_location_summary</code> <code>/3</code>	580-59 4	Prints round-trip distance, route, battery cost and weight for one camp.
<code>view_delivery_details</code> <code>/0</code>	596-60 6	Option 1: forall prints a summary for every pending camp.

How it works, step by step

1. `block_road` checks the input, then `known_road`, then that the road is not blocked already, then `assertz(blocked(A,B))`.
2. `valid_move` (Member 2) reads `blocked/2`, so every algorithm avoids the road at once.
3. `unblock_road` uses `retractall` for both directions.

4. Option 1: for every pending camp, A* finds the way out and the way back; costs and km are added and battery_percent converts the energy.
5. forall runs show_location_summary for every available camp without keeping variable bindings.

Code of this part (with line numbers)

```

538 | %----- Block or unblock a road -----
539 |
540 | block_road :-
541 |     nl, write('Enter Road to Block (example: Ratnapura. Kandy.): '),
542 |     read_atom_input(A), read_atom_input(B),
543 |     ( (var(A) ; var(B))
544 |     -> nl, write('Invalid input: could not read those location
names. '), nl
545 |     ; known_road(A, B)
546 |     -> ( (blocked(A, B) ; blocked(B, A) )
547 |     -> nl, write('That road is already blocked. '), nl
548 |     ; assertz(blocked(A, B)),
549 |     nl, write('Road blocked: '), write(A-B), nl
550 |     )
551 |     ; nl, write('Unknown road. No road was blocked. '), nl
552 |     ).
553 |
554 | known_road(A, B) :- edge(A, B, _).
555 | known_road(A, B) :- edge(B, A, _).
556 |
557 | unblock_road :-
558 |     nl, write('Enter Road to Unblock (example: Ratnapura. Kandy.): '),
559 |     read_atom_input(A), read_atom_input(B),
560 |     ( (var(A) ; var(B))
561 |     -> nl, write('Invalid input: could not read those location
names. '), nl
562 |     ; known_road(A, B),
563 |     (blocked(A, B) ; blocked(B, A))
564 |     -> retractall(blocked(A, B)),
565 |     retractall(blocked(B, A)),
566 |     write('Unblocked: '), write(A-B), nl
567 |     ; nl, write('No such blocked road found. '), nl
568 |     ).
569 |
570 | show_blocked :-
571 |     nl, write('Currently blocked roads: '), nl, nl,
572 |     (blocked(_, _) -> list_blocked ; write('None')), nl).
573 |
574 | list_blocked :- forall(blocked(A,B), (write(A-B), nl)).
575 |
576 |
577 |
578 | %----- Options 1 and 2: single deliveries -----
579 |
580 | show_location_summary(Current, Loc, Weight) :-
581 |     ( astar(Current, Loc, OutPath, OutCost),

```

```

582 |         astar(Loc, Current, ReturnPath, ReturnCost)
583 |         -> TotalCost is OutCost + ReturnCost,
584 |         path_distance_km(OutPath, OutKm),
585 |         path_distance_km(ReturnPath, ReturnKm),
586 |         TotalKm is OutKm + ReturnKm,
587 |         battery_percent(TotalCost, Pct),
588 |         format('Location: ~w~n', [Loc]),
589 |         format('  Distance: ~w km (round trip)~n', [TotalKm]),
590 |         format('  Route: ~w~n', [OutPath]),
591 |         format('  Battery Cost: ~w% (round trip)~n', [Pct]),
592 |         format('  Package Weight: ~wkg~n', [Weight])
593 |         ; format('Location: ~w | No valid round-trip path from ~w.~n',
[Loc, Current])
594 |     ).
595 |
596 | view_delivery_details :-
597 |     current_location(Current),
598 |     nl, write('--- Delivery Locations (goods to be delivered) ---'),
nl,
599 |     format('(Calculated from current location: ~w, round trip)~n',
[Current]),
600 |     ( \+ available_location(_, _)
601 |     -> write('All relief camps have been served. No pending
deliveries.'), nl
602 |     ; forall(
603 |         available_location(Loc, Weight),
604 |         show_location_summary(Current, Loc, Weight)
605 |     )
606 |     ).

```

Real output: options 3, 5 and 4 (block, list, unblock)

```

Road blocked: Colombo-Kandy
Colombo-Kandy
That road is already blocked.
Ratnapura. Kandy.): Unblocked: Colombo-Kandy

```

Real output: option 1 (first camps)

```

--- Delivery Locations (goods to be delivered) ---
(Calculated from current location: Colombo, round trip)
Location: Hambantota
  Distance: 316 km (round trip)
  Route: [Colombo,Ratnapura,Hambantota]
  Battery Cost: 30% (round trip)
  Package Weight: 15kg

```

Member 6: Option 2: deliver to a selected location

Name	<i>A.W.A.D. Thisara Dimuth Abesekara</i>
Registration no.	524654961
Line range	608-682 (67 code lines)
Menu options / functions	Option 2

Contribution summary

Option 2: reading the destination, checking it, calculating the round trip, checking the battery, asking for confirmation and completing the delivery. Also wrote the yes/no question function used by option 6.

Implemented functions

Function / facts	Lines	What it does
<code>deliver_to_location/0</code>	608-628	Reads the destination, checks it and computes the outward and return routes with A*.
<code>review_and_confirm/7</code>	630-655	Shows the delivery details, checks the battery and asks for confirmation.
<code>ask_confirmation/1</code>	659-668	Asks yes/no and repeats the question if the answer is not yes or no. Also used by option 6.
<code>execute_delivery/5</code>	671-682	Reduces the battery, marks the camp delivered and prints the result.

How it works, step by step

1. `deliver_to_location` is a chain of if-then-else checks: unreadable input, valid camp, already delivered, invalid location.

2. Round trip = A* from the base to the camp plus A* back; costs and km are added and battery_percent converts the energy.
3. review_and_confirm compares the percentage with the battery; if it is enough, ask_confirmation is called, otherwise "DELIVERY DENIED".
4. ask_confirmation accepts yes/y/no/n in any letter case and asks again for anything else.
5. execute_delivery swaps the battery fact (retract then assertz) and adds delivered(Target). The drone returns to the base, so current_location never changes.

Code of this part (with line numbers)

```

608 | deliver_to_location :-
609 |     findall(L, available_location(L, _), Locations),
610 |     (   Locations == []
611 |     -> nl, write('All relief camps have been served. Nothing left to
deliver. '), nl
612 |     ;   current_location(Current),
613 |     nl, write('Available locations: '), write(Locations), nl,
614 |     write('Enter destination location (end with a period, e.g. Kandy.):
'),
615 |     read_atom_input(Target),
616 |     (   var(Target)
617 |     -> nl, write('Invalid input: could not read that location name. '),
nl
618 |     ;   available_location(Target, TargetWeight)
619 |     -> (   astar(Current, Target, OutPath, OutCost),
620 |         astar(Target, Current, ReturnPath, ReturnCost)
621 |     -> review_and_confirm(Current, Target, OutPath, OutCost,
ReturnPath, ReturnCost, TargetWeight)
622 |     ;   format('No valid round-trip path between ~w and ~w.
Delivery not possible.~n', [Current, Target])
623 |     )
624 |     ;   delivered(Target)
625 |     -> nl, format('~w has already received its delivery.~n', [Target])
626 |     ;   nl, write('Invalid delivery location. '), nl
627 |     )
628 |     ).
629 |
630 | review_and_confirm(Current, Target, OutPath, OutCost, ReturnPath,
ReturnCost, Weight) :-
631 |     TotalCost is OutCost + ReturnCost,
632 |     battery_percent(TotalCost, Pct),
633 |     battery_level(Bat),
634 |     combine_paths(OutPath, ReturnPath, FullPath),
635 |     path_distance_km(OutPath, OutKm),
636 |     path_distance_km(ReturnPath, ReturnKm),
637 |     TotalKm is OutKm + ReturnKm,

```

```

638 |         nl, write('--- Delivery Details (round trip) ---'), nl,
639 |         format('Location: ~w~n', [Target]),
640 |         format('  Distance: ~w km (round trip)~n', [TotalKm]),
641 |         format('  Route: ~w~n', [OutPath]),
642 |         format('  Battery Cost: ~w% (round trip)~n', [Pct]),
643 |         format('  Package Weight: ~wkg~n', [Weight]),
644 |         format('  Battery Available: ~w%~n', [Bat]),
645 |         ( Pct <= Bat
646 |         -> Remaining is Bat - Pct,
647 |         format('  Battery After Delivery: ~w%~n', [Remaining]),
648 |         nl,
649 |         ( ask_confirmation('Confirm this delivery? (yes. / no.): ')
650 |         -> execute_delivery(Current, Target, FullPath, Pct, Weight)
651 |         ; nl, write('Delivery cancelled. No battery used. '), nl
652 |         )
653 |         ; nl, write('*** DELIVERY DENIED ***'), nl,
654 |         format('Insufficient battery for this round trip. Required: ~w%,
Available: ~w%~n', [Pct, Bat])
655 |         ).
656 |
657 |
658 |
659 | ask_confirmation(Prompt) :-
660 |     write(Prompt),
661 |     read_answer(Answer),
662 |     ( memberchk(Answer, [yes, y])
663 |     -> true
664 |     ; memberchk(Answer, [no, n, end_of_file])
665 |     -> fail
666 |     ; write('Please answer yes. or no. '), nl,
667 |     ask_confirmation(Prompt)
668 |     ).
669 |
670 |
671 | execute_delivery(Current, Target, Path, Pct, Weight) :-
672 |     battery_level(Bat),
673 |     NewBat is Bat - Pct,
674 |     retract(battery_level(Bat)),
675 |     assertz(battery_level(NewBat)),
676 |     % The drone flies out, delivers, and flies back;
current_location(Current)
677 |     % stays as-is since the round trip ends where it started.
678 |     assertz(delivered(Target)), % Mark the location as served.
679 |     nl, write('*** DELIVERY APPROVED ***'), nl,
680 |     format('Route Taken (round trip): ~w~n', [Path]),
681 |     format('Battery Used: ~w% | Battery Remaining: ~w%~n', [Pct,
NewBat]),
682 |     format('Delivered ~wkg of supplies to ~w and returned to ~w.~n',
[Weight, Target, Current]).

```

Real output: option 2, deliver to Kandy

```

--- Delivery Details (round trip) ---
Location: Kandy
  Distance: 190 km (round trip)
  Route: [Colombo,Kandy]

```

```
Battery Cost: 18% (round trip)
Package Weight: 20kg
Battery Available: 100%
Battery After Delivery: 82%
```

```
Confirm this delivery? (yes. / no.):
```

```
*** DELIVERY APPROVED ***
```

```
Route Taken (round trip): [Colombo,Kandy,Colombo]
```

```
Battery Used: 18% | Battery Remaining: 82%
```

```
Delivered 20kg of supplies to Kandy and returned to Colombo.
```

Real output: repeated and wrong locations

```
Kandy has already received its delivery.
```

```
Invalid delivery location.
```